Lab2 实验报告

姓名：王思博 学号：25300190047

第一关：

（1）关卡分析：

决定关卡是否通过的关键指令为：

```
mov %r12,%rdi
call p1_compute
cmp %eax,-0x14(%rbp)
sete %al
```

其中，r12 保存 challenge。调用 p1_compute 前，程序将 challenge 放入第一个参数寄存器 rdi。函数返回后，eax 保存计算结果，-0x14(%rbp) 保存输入的整数。两者相等时，sete 将 al 置为 1，表示通过。

因此，本关需要还原 p1_compute(challenge) 的计算过程。

（2）计算关系推导

p1_compute 首先复制并移位 challenge：

```
mov %rdi,%rax
mov %rdi,%rsi
mov %rdi,%rdx
shr $0x20,%rax
shr $0x10,%rsi
mov %eax,%r8d
mov %eax,%edi
```

据此定义三个 32 位无符号整数：

```
uint32_t high = (uint32_t)(challenge >> 32);
uint32_t mid  = (uint32_t)(challenge >> 16);
uint32_t low  = (uint32_t)challenge;
```

程序实际依次调用 p1_step1、p1_step0 和 p1_step2。

第一次调用 p1_step1(high, mid)。其关键指令为：

```
sub %esi,%edi
lea 0x57ad(%rdi,%rdi,2),%eax
```

sub 计算第一个参数减去第二个参数，lea 将差乘以 3 并加上常数，得到：

```
uint32_t t1 = 3u * (high - mid) + 0x57adu;
```

第二次调用前的参数准备为：

```
mov %r8d,%esi
mov %edx,%edi
mov %eax,%ecx
call p1_step0
```

此时 r8d 保存 high，edx 保存 low，而 ecx 用于备份 t1，因此实际调用为 p1_step0(low, high)。该函数执行：

```
xor $0x370b,%esi
lea (%rsi,%rdi,2),%eax
```

对应关系为：

```
uint32_t t0 = (high ^ 0x370bu) + 2u * low;
```

最后，程序调用 p1_step2(t0, t1)：

```
mov %ecx,%esi
mov %eax,%edi
call p1_step2
```

该函数的关键指令为：

```
lea 0xa34e(%rsi),%eax
xor %edi,%eax
```

因此最终答案为：

```
uint32_t answer = (t1 + 0xa34eu) ^ t0;
```

以上运算均按低 32 位保留结果。实现时使用 uint32_t，以保持与汇编中 32 位运算一致的效果。

（3）运行结果

```
sibo_wang@Sibo:~/ics-2026-bomblab$ ./defuser --seed 42 --phase 1 | ./bomb --seed 42 --phase 1
第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0x28efe333b266f103
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

sibo_wang@Sibo:~/ics-2026-bomblab$
```

```
sibo_wang@Sibo:~/ics-2026-bomblab$ ./defuser --seed 314159 --phase 1 | ./bomb --seed 314159 --phase 1
第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0xd85dda4696932a2d
Input (unsigned decimal integers):
Phase 1 defused.

[1 已解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

运行后炸弹成功被 defused，在更换 seed 314159 后也能够解除炸弹，通过第一关。

第二关：

（1）输入与初始值：

phase_02 调用 valid_u32_fields 时，将第二个参数设为 6，并在 sscanf 返回后检查：

```
cmp $0x6,%eax
jne 4020a5
```

因此，本关要求输入六个无符号整数。它们连续存放在栈上，每个占 4 字节，记为 a[0] 至 a[5]。

第一个数的检查如下：

```
mov %r13,%rax

mov -0x40(%rbp),%esi

shr $0x13,%rax

and $0x7f,%eax

cmp %eax,%esi

jne 4020a5
```

r13 保存 challenge，esi 读取输入的第一个数。程序将 challenge 右移 19 位，再保留最低 7 位，与输入比较。因此：

```
a[0] = (uint32_t)((challenge >> 19) & 0x7f);
```

（2）循环关系

循环开始前，rdx 指向 a[0]，esi 保存 a[0]，edi 初始化为 7。

程序通过以下指令构造移位数：

```
mov $0x4,%r8d

sub %edx,%r8d
```

循环中执行：

```
lea (%r8,%rdx,1),%ecx
```

按低 32 位计算，第一次得到 ecx = 4。此后 rdx 每轮增加 4，而 r8 保持不变，因此移位数依次为 4、8、12、16、20。

循环中的核心计算为：

```
mov %r13,%rax

add %esi,%esi

shr %cl,%rax

and $0x3f,%eax

xor %esi,%eax

mov 0x4(%rdx),%esi

add %edi,%eax

cmp %eax,%esi
```

`jne 4020a5`

其中，`add %esi,%esi` 将前一个数乘以 2；右移和掩码操作从 `challenge` 中取出 6 位；随后进行异或并加上 `edi`。`mov 0x4(%rdx),%esi` 读取下一个输入整数，与计算结果比较。

每轮结束时：

`add $0x7,%edi`

`add $0x4,%rdx`

`cmp $0x2a,%edi`

`jne 40210d`

加数依次为 7、14、21、28、35。检查完 `a[5]` 后，`edi` 增加到 42，循环结束。

因此，恢复出的递推关系为：

```
a[0] = (uint32_t)((challenge >> 19) & 0x7f);
for (unsigned int i = 1; i < 6; ++i) {
    uint32_t part =
        (uint32_t)((challenge >> (4 * i)) & 0x3f);
    a[i] = (part ^ (2u * a[i - 1])) + 7u * i;
}
```

（3）实验结果





在 defuser.c 的 solve_phase_2 中，先计算 values[0]，再通过循环依次计算其余五个整数，设置 count = 6，由 write_answer 输出。在两个不同的 seed 下都显示炸弹被拆除，本关通过。

第三关：

1. 输入及参数计算

`phase_03` 调用 valid_u32_fields 时，第二个参数为 2，并检查 `sscanf` 的返回值是否为 2，因此本关要求输入两个无符号整数。

第一个数的检查如下：

```
mov %r12d,%edi
and $0x7,%edi
cmp %edi,-0x18(%rbp)
```

`r12` 保存 challenge。程序取其最低三位，与第一个输入比较，因此：

```
uint32_t index = (uint32_t)(challenge & 7u);
```

随后为 `p3_dispatch` 准备参数：

```
mov %r12,%rax

mov %r12,%rsi

shr $0x3,%rax

shr $0x20,%rsi

xor %eax,%esi

call p3_dispatch
```

第一个参数 edi 仍为 index，第二个参数为：

```
uint32_t x = (uint32_t)(challenge >> 32)

           ^ (uint32_t)(challenge >> 3);
```

p3_dispatch 的返回值与第二个输入比较，相等时通过。

(2)跳转和分支对应

p3_dispatch 的关键指令为：

```
mov %edi,%eax

mov %esi,%edi

cmp $0x7,%eax

ja 402225

notrack jmp *0x4057e0(,%rax,8)
```

程序将分支编号保存到 eax，将 x 放入 edi，作为后续分支函数的参数。编号在 0～7 范围内时，从跳转表读取目标地址并跳转。每个地址占 8 字节，因此使用 index × 8 计算表项偏移。

使用 GDB 查看跳转表：

```
x/8gx 0x4057e0
```

将表项与对应地址的跳转指令结合，分支函数名的数字不能直接作为输入编号，必须根据跳转表确定对应关系。

（3）分支计算关系

各分支通过移位、异或、加法以及 lea 完成计算。例如，p3_case_6 的关键指令为：

```
mov %edi,%eax
shr $1,%edi
xor $0x481,%eax
add %edi,%eax
```

它先保留原始 x，将另一份副本右移一位，再计算：

```
(x ^ 0x481u) + (x >> 1)
```

p3_case_1 的关键指令为：

```
lea 0x6a7(%rdi,%rdi,4),%eax
```

该指令计算 `0x6a7 + x + 4*x`，即 `5*x + 0x6a7`。

结合跳转表，第二个答案的完整计算关系如下：

index	第二个答案
0	`(x ^ 0x481u) + (x >> 1)`
1	`(2u * x) ^ 0x2db4u`
2	`(x ^ 0x9849u) + (x >> 1)`
3	`5u * x + 0x6a7u`
4	`5u * x + 0x1806fu`
5	`5u * x + 0x98deu`
6	`(4u * x) ^ 0x7720u`
7	`5u * x + 0xbf9fu`

在 solve_phase_3 中使用 switch 实现上述分支，将 index 和计算结果分别写入 values[0]、values[1]，设置 count = 2。使用 uint32_t 保持与汇编中低 32 位结果一致。

（4）实验结果





在两个 seed 下都显示 defused，本关通过。

第四关：

1. 输入要求与参数计算

phase_04 调用 valid_u32_fields 时，第二个参数为 1，并检查 sscanf 的返回值是否为 1，说明本关要求输入一个无符号整数。

r12 保存 challenge。调用 p4_walk 前，程序通过移位和异或准备参数：

```
mov %r12,%rcx

mov %r12,%rax

mov %r12d,%edi

xor %esi,%esi

shr $0x20,%rax

shr $0x7,%rcx

and $0x7f,%edi

mov $0x7f,%edx

xor %eax,%ecx

call p4_walk
```

根据参数传递所使用的寄存器，可将调用关系还原为：

```
uint32_t a = (uint32_t)(challenge & 0x7fu);

uint32_t b = (uint32_t)(challenge >> 7)

           ^ (uint32_t)(challenge >> 32);


uint32_t answer = p4_walk(a, 0u, 127u, b);
```

其中，a 是区间内需要寻找的数，范围为 0 ~ 127；b 参与返回值计算。

最后，程序将函数返回值与输入进行比较，相等时通过本关。

2. 区间中点与递归结束条件

p4_walk 的四个参数依次为 a、low、high、b。

函数开始时执行：

```
mov %edi,%eax
mov %edx,%edi
sub %esi,%edi
shr $1,%edi
add %esi,%edi
```

程序先将 a 保存到 eax，然后计算区间中点：

```
uint32_t mid = low + ((high - low) >> 1);
```

随后比较 mid 与 a。如果相等，则执行：

```
lea 0x3(%rdi),%eax
xor %ecx,%eax
ret
```

因此递归结束条件为：

```
if (a == mid) {
    return (mid + 3u) ^ b;
}
```

3. 递归分支及返回值处理

当 a 不等于中点时，程序先保存当前层的：

```
mid + b
```

随后通过以下指令判断方向：

```
cmp %edi,%eax
jb 4022f0
```

此时 eax = a、edi = mid。jb 表示无符号数比较中的“小于则跳转”，因此：

- 当 a < mid 时，在 [low, mid-1] 中递归查找。
- 当 a > mid 时，在 [mid + 1, high] 中递归查找。

左半边递归返回后，程序将子调用结果乘以 5，再异或当前层的 mid+b 和常数 7：

```
uint32_t r = p4_walk(a, low, mid - 1u, b);
return (5u * r) ^ (mid + b) ^ 7u;
```

右半边的处理类似，最后异或的常数为 1：

```
uint32_t r = p4_walk(a, mid + 1u, high, b);
return (5u * r) ^ (mid + b) ^ 1u;
```

因此，该函数先不断缩小区间，直到中点等于 a，再逐层处理子调用的返回值。

（4）实验结果





在两个 seed 下，炸弹都显示 defused，本关通过。

第五关：

1. 输入要求

phase_05 调用 valid_u32_fields 时，第二个参数为 2，并检查 sscanf 是否成功读取两个整数，因此本关要求输入两个无符号整数。

解析后，第一个输入存放在 rbp - 0x1c，第二个输入存放在 rbp - 0x18。

随后执行：

```
lea -0x14(%rbp),%rsi

mov %r12,%rdi

call p5_traverse
```

其中，r12 保存 challenge，第二个参数是用于接收结果的局部变量地址。调用关系可表示为：

```
uint32_t b;

uint32_t a = p5_traverse(challenge, &b);
```

程序要求第一个输入等于返回值 a，第二个输入等于通过地址写出的值 b。

2. 遍历过程

函数通过 challenge 的最低三位选择起始节点：

```
and $0x7,%eax

mov 0x405b20(,%rax,8),%rdx
```

对应：

```
uint32_t index = (uint32_t)(challenge & 7u);
```

地址 0x405b20 处存放八个节点指针，每项占 8 字节。使用 GDB 查看：

```
x/8gx 0x405b20
```

函数还通过以下操作计算向后移动的步数：

```
uint32_t steps = (uint32_t)((challenge >> 3) & 3u);
```

steps 的范围为 0～3。起始节点会先被处理，随后再向后移动 steps 次，因此总共处理 steps+1 个节点。

3. 节点结构与数据恢复

根据节点访问指令：

```
mov (%rdx),%rdx
mov 0x8(%rdx),%esi
add 0xc(%rdx),%eax
```

可推断每个节点占 16 字节，其布局为：

偏移	内容	大小
0	下一个节点的指针	8 字节
8	用于异或的 value	4 字节
12	用于加法的 add	4 字节

将起始指针表转换为节点编号，得到：heads[8] = {9, 4, 5, 6, 11, 1, 7, 0};

4. 计算规则及实现思路

函数首先根据 challenge 计算初始值：

```
uint32_t x = (uint32_t)(challenge >> 6)
           ^ (uint32_t)(challenge >> 32)
           ^ 0x486cu;
```

每处理一个节点，执行：

```
x = (3u * x + node_add[node]) ^ node_value[node];
```

同时，将当前节点的 value 写入第二个答案的存放位置。因此，遍历结束后：第一个答案为最终的 x，第二个答案为最后一个被处理节点的 value。

5.实验结果





在两个不同的 seed 下炸弹都 defused 本关通过。

第六关：

1. 输入要求与检查逻辑

phase_06 调用 valid_u32_fields 时，第二个参数为 1，随后检查 sscanf 的返回值是否为 1，说明本关要求输入一个无符号整数。

输入存放在 rbp - 0x14。程序将 challenge 作为参数调用 p6_dispatch，并将其返回值与输入比较：

```
mov %r12,%rdi

call p6_dispatch

cmp %eax,-0x14(%rbp)

sete %al
```

因此，本关的正确答案为 p6_dispatch(challenge) 的返回值。

2. 参数提取与两次间接调用

分析 p6_dispatch 可知，程序先计算初始参数：

```
uint32_t x = (uint32_t)(challenge >> 6)

           ^ (uint32_t)(challenge >> 32);
```

第一次调用使用：

```
and $0x7,%eax

call *0x405dc0(,%rax,8)
```

因此，第一次调用的表下标为：

```
uint32_t a = (uint32_t)(challenge & 7u);
```

程序从函数指针表中读取第 a 项，并将 x 作为参数调用对应函数。

第一次调用返回后，程序将返回值异或 0x326c，作为第二次调用的输入。

第二次调用的字节偏移由 challenge & 0x38 得到。每个函数指针占 8 字节，因此对应的表下标为：

```
uint32_t b = (uint32_t)((challenge >> 3) & 7u);
```

第二次调用结束后，程序将结果加上 0xb9bf，作为最终返回值。

完整计算关系为：

```
uint32_t r = table[a](x);

r = table[b](r ^ 0x326cu);

uint32_t answer = r + 0xb9bfu;
```

3.函数指针和调用

结合表项地址和函数指令，得到以下对应关系：

表下标	函数	运算规则
0	p6_op_3	(x ^ 0x7438u) + (x << 7)
1、2	p6_op_5	32 位循环右移 2 位
3、4	p6_op_2	(4u * x + 0xb375u) ^ (5u * x)
5	p6_op_0	(x ^ (x >> 5)) + 0x9e05u
6	p6_op_4	32 位循环左移 7 位，再异或 0x6f45u
7	p6_op_1	(x + 0x2f02u) ^ (x >> 7)

其中，下标 1 和 2 指向同一个函数，下标 3 和 4 也指向同一个函数，因此八个表项实际对应六个不同函数。

循环移位使用以下 C 表达式实现：

```
/* 循环右移 2 位 */

(x >> 2) | (x << 30)

/* 循环左移 7 位 */

(x << 7) | (x >> 25)
```

与普通移位不同，循环移位会将移出一端的位补到另一端。

4.实验结果

```
sibo_wang@Sibo:~/ics-2026-bomblab$ ./defuser --seed 314159 --phase 6 | ./bomb --seed 314159 --phase 6
                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 当前]

------------------------------------------------------------

单关调试：只记录本次通过的关卡，不播放完整结局。

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0xc1a32713817bbecc
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]
```

```
sibo_wang@Sibo:~/ics-2026-bomblab$ ./defuser --seed 42 --phase 6 | ./bomb --seed 42 --phase 6
                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 当前]

------------------------------------------------------------

单关调试：只记录本次通过的关卡，不播放完整结局。

第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0x37e9671c45376d5d
Input (unsigned decimal integers):
Phase 6 defused.

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 已解除]
```

在两个不同的 `seed` 下都显示炸弹 `defused`，本关通过。

隐藏关：

1. 隐藏关定位

使用以下命令查找相关符号：

```
nm -n ./bomb | grep -Ei 'secret|hidden|phase|unlock'
```

发现 secret_gate、secret_copy、secret_phase 和 secret_mix 等函数。随后通过 objdump 查看其反汇编，分析入口条件和答案计算方式。

2. 入口字符串格式

secret_gate 首先检查输入字符串长度：

```
call strlen

sub $0x24,%rax

cmp $0x7f,%rax

jbe 402720
```

该无符号比较要求字符串长度在 36～163 字节之间。

程序通过 secret_copy 将字符串逐字节复制到局部缓冲区，然后检查相对缓冲区起点偏移 32 字节处的内容：

```
cmpl $0x4e45504f,-0xb0(%rbp)
```

由于 x86 使用小端存储，0x4e45504f 对应字符 "OPEN"。

之后，程序将缓冲区起点偏移 36 字节处的字符串传给 secret_phase。因此，输入可以构造为：

32 个 A + OPEN + 空格 + 三个无符号十进制整数

3. challenge 与 context 的来源

分析 main 中调用 secret_gate 附近的指令，发现程序先调用 transcript_context，再以关卡编号 7 调用 phase_challenge。

因此，隐藏关计算依赖两个参数：

challenge：第七关的 challenge。

context：前六关答案经过 transcript_context 计算得到的上下文。

transcript_context 依次提取前六关答案中的整数，并结合关卡编号、整数位置进行 64 位混合。初始状态为：

```
uint64_t state = UINT64_C(0x243f6a8885a308d3);
```

其基础混合函数为：

```
static uint64_t context_mix_local(uint64_t x) {
    x = (x ^ (x >> 30)) * UINT64_C(0xbf58476d1ce4e5b9);
    x = (x ^ (x >> 27)) * UINT64_C(0x94d049bb133111eb);
    return x ^ (x >> 31);
}
```

每关开始时，将状态与关卡编号异或后混合；每读取一个整数，再将状态与整数数值及位置标记异或后混合。位置标记为：

```
((uint64_t)phase << 32) | position
```

其中关卡编号从 1 开始，关卡内的整数位置从 0 开始。

4. 三个答案的计算

secret_phase 要求成功读取三个无符号整数。

首先定义：

```
uint32_t a = (uint32_t)(context & 3u);
uint32_t x = (uint32_t)(challenge >> 32)
           ^ (uint32_t)challenge;
uint32_t y = (uint32_t)(context >> 32)
           ^ (uint32_t)context;
uint32_t z = (a * 0x802eu)
           ^ (uint32_t)(context >> 16);
```

第一个答案为 a。

分析 secret_mix：

```
mov %edi,%eax
```

```
xor $0x802e,%edi

shr $1,%eax

xor $0x2976,%eax

add %edi,%eax
```

可得：`mix(t) = ((t >> 1) ^ 0x2976u) + (t ^ 0x802eu);`

因此第二个答案为：`uint32_t b = mix(x ^ y) ^ z;`

第三个答案为：`uint32_t c = b ^ (y + a + 0x2976u + (x >> 1));`

上述计算均使用 `uint32_t`，与汇编中的 32 位运算保持一致。

5.实验结果

```
sibo_wang@Sibo:~/ics-2026-bomblab$ {
    ./defuser --seed 42
    ./defuser --secret --seed 42
} | ./bomb --seed 42
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

```
sibo_wang@Sibo:~/ics-2026-bomblab$ {
    ./defuser --seed 314159
    ./defuser --secret --seed 314159
} | ./bomb --seed 314159
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

Honor · 观测台旧备份

林夏准备关闭档案列表，忽然指着最下面一项：‘这里还有个备份。’
文件夹叫‘留给后来的人’，署名是 sky 和许遥。
扉页写着：‘交接完成后，如果你还有时间，可以到观测台坐坐。’
这份旧档案不参与城市的运行。你们沿着楼梯登上观测台，
桌上的阅读器仍在等待旧程序认可的输入。
城市已经安全了。是否继续读下去，由你们自己决定。

Secret phase defused.
[SAFE] 城市照常运行。附加档案已恢复。
```

在两个不同的 seed 下都显示 secret phase defused，Honor 关卡通过。